

ChatXPT Turns Livestream Chat Into Playable Sidequests

- AI stream director for game streamers on Twitch.
- Converts gameplay moments, audience behaviour, and streamer preferences into exactly three safe sidequest options.
- Viewers vote; the winning quest becomes an OBS broadcast moment.

Evidence labels product promise [diagnostic]; end-to-end live proof [unknown].

Asset note Hero visual from final demo showing Studio, viewer vote, and overlay in one composition; avoid implying live evidence until manifest entry exists.

Livestream Chat Has Energy, But Little Structured Agency

- Viewers suggest ideas, react, and spam chat, but streamers cannot safely process all of it while playing.
- Static polls and fixed rewards do not adapt to the current match, audience mood, or streamer limits.
- The missed opportunity is not more chat volume; it is safe, contextual participation.

Evidence labels problem framing [diagnostic]; participant validation [unknown].

Asset note Optional sanitized observation quote or workflow screenshot after docs/research/PRODUCT-VALIDATION.md receives real participant evidence.

The Primary Users Are Streamers, Viewers, and Moderators

- Streamers need hands-on control, persistent setup, safety limits, and low distraction.
- Viewers want a fast way to influence the stream without leaving Twitch.
- Moderators and stream teams need compact controls, status clarity, and emergency recovery.
- Judges and maintainers need proof labels that distinguish real, fixture, diagnostic, and unknown states.

Evidence labels target users from product brief and role plans [diagnostic]; usability observations [unknown].

Asset note Studio setup/status screenshot for streamer; Twitch/hosted viewer screenshot for viewer; compact Live Config screenshot for moderator.

The Product Promise Is A Closed Participation Loop

- ChatXPT watches real gameplay and audience signals through authorised adapters.
- Role 2 intelligence proposes exactly three structured candidate quests.
- Role 3 validates safety, feasibility, timing, fallback, voting, progress, and outcome rules.
- Role 1 persists one authoritative revision and broadcasts safe views to Studio, viewers, and OBS.

Evidence labels architecture/contracts [diagnostic]; Role 5 visual fixture surfaces [fixture]; live closed loop [unknown].

Asset note One clean architecture diagram or final golden-workflow sequence with session/cycle revision visible in evidence notes.

ChatXPT Stays Twitch-First Without Becoming Twitch-Locked

- Twitch is the only implemented MVP platform.
- Twitch Extension is the primary viewer surface; hosted Quest Board and 1/2/3 Twitch-chat voting are fallbacks.
- OBS Browser Source is visual output only, not the configuration or voting surface.
- YouTube, Discord, TikTok, and other platforms remain disabled Coming Soon options.

Evidence labels accepted product scope [diagnostic]; Twitch test registration/runtime [unknown].

Asset note Twitch Extension hosted/local test screenshots after allowlisted test evidence; hosted board fallback screenshot already has fixture evidence.

AI Contributes Context, But Determinism Holds Authority

- AI contribution is scoped to gameplay/audience intelligence and candidate generation.
- The product never requires paid model usage for the MVP.
- Provider output must be structured, validated, and routed through Role 3 before viewers see it.
- If a provider is slow, unavailable, malformed, or refused, credential-free algorithms and deterministic fallback continue the workflow.

Evidence labels

provider fallback tests and strategy [diagnostic]; live free-provider run [unknown]; algorithmic path on real inputs [unknown].

Asset note

Studio/provider status screenshot showing provider, algorithmic, and fallback labels after Role 1 promotes canonical UI-X09 fixtures or live evidence exists.

Gameplay Understanding Is Honest About Confidence And Unknowns

- OBS Virtual Camera is the accepted MVP capture path for raw game frames.
- Universal visual signals cover broad activity, quiet, and transitions across action games.
- Calibrated HUD facts are claimed only for configured games and evidenced regions.
- Unsupported or low-confidence facts stay unknown; they are not guessed from expectations.

Evidence labels	extraction boundary and synthetic tests [diagnostic]; synthetic visual/OCR tests [fixture]; real OBS frame extraction [unknown].
------------------------	--

Asset note	Privacy-reviewed OBS Virtual Camera frame capture with separated annotation overlays; one selected demo-game calibrated fact if proven.
-------------------	---

The Architecture Uses One Runtime Source Of Truth

- Next.js and TypeScript provide one product shell with role-owned public modules.
- Zod schemas define versioned platform-neutral contracts for signals, profiles, sessions, candidates, votes, quest state, views, commands, and errors.
- Role 1's orchestrator authenticates commands, checks revisions, deduplicates IDs, persists atomically, and broadcasts role-specific views.
- Role modules integrate through public ports; private Twitch, OBS, provider, Supabase, and UI payloads stay outside Core.

Evidence labels source/contracts/tests [diagnostic]; live multi-client authority [unknown].

Asset note Compact data-flow diagram plus a later screenshot/log proving the same session and quest-cycle revision across Studio, two viewers, persistence, and OBS.

The Quest Engine Makes Safety And Flow Deterministic

- It validates unsafe, illegal, humiliating, wagering, privacy-violating, unsupported, repetitive, stale, and impossible quests.
- It opens a 30-second vote using authoritative time and rejects late votes.
- It resolves majority, ties, zero-vote no-activation, cancellation, skip, failure, success, expiry, cooldown, points, and hype through deterministic rules.
- Human-facing surfaces show allowed actions; they do not calculate winners, rewards, or lifecycle transitions.

Evidence labels engine and validation tests [fixture]; live quest cycle [unknown].

Asset note Test result summary or demo screenshots for proposed, voting, active, terminal, tie, zero-vote, and emergency states.

Participation Converges Through One Private Service

- Twitch Extension, hosted Quest Board, and Twitch-chat fallback all submit to the same authoritative vote ledger.
- The first accepted vote per viewer and quest cycle is final.
- Shared broadcasts remove private vote receipts and personal points.
- Reconnecting viewers recover only their own accepted vote and session points through a private recovery seam.

Evidence labels memory/Supabase source and tests [diagnostic]; Role 5 viewer fixture screenshots [fixture]; real two-viewer vote [unknown].

Asset note Two isolated viewer clients voting in one real or memory-backed run; include duplicate/late vote evidence only when exercised.

Streamer Studio Keeps Control With Minimal Live Burden

- Studio is the full management surface for Twitch setup, OBS capture, profile preferences, safety, game context, health, diagnostics, history, and advanced
- Twitch Live Config is the compact stream-time control surface inside Twitch.
- Current Role 4 work includes a design-system handoff, setup shell, readiness/status presentation, and render-only status modules.
- Authoritative connection, profile save, session wiring, and Live Config controls still depend on Role 1 gateway handoffs.

Evidence labels Role 4 source/tests [diagnostic]; final integrated Studio [unknown].

Asset note Desktop and narrow Studio screenshots for setup, returning readiness, generated quests, recovery, and compact Live Config.

Viewer And Overlay Surfaces Show The Shared Payoff

- Viewers see exactly three understandable options, vote acknowledgement, tallies, countdown, active quest, progress, results, hype, and recovery states.
- Hosted board fallback supports desktop/mobile participation without a separate account.
- Twitch-chat fallback explains the 1/2/3 voting path without putting chat parsing or sending inside Role 5.
- OBS overlay stays read-only and low-distraction for the broadcast.

Evidence labels

Role 5 fixture-rendered screenshots in manifest [fixture]; Twitch Extension identity/hosted access/realtime/OBS proof [unknown].

Asset note

Existing fixture assets can illustrate current UI; final deck needs real or memory-backed same-revision evidence before claiming integration.

Expected Impact Is Measured As Participation, Not Vanity

- Principal metric: viewer participation rate, the share of active viewers who cast an accepted vote.
- Supporting measures: quest completion, setup time, response latency, fallback/reconnect success, and streamer control burden.
- Product hypotheses remain open until recorded streamer/viewer observations and executed runs exist.
- The deck should describe impact as measurable hypotheses unless the manifest records results.

Evidence labels accepted KPI decision [diagnostic]; measured impact [unknown].

Asset note A simple metrics snapshot or table after real or memory-backed rehearsal: viewers present, accepted votes, completion, latency, reconnect/fallback outcome.

Technical Choices Prioritise Trust, Privacy, And Recoverability

- Supabase Free is the accepted authoritative persistence/realtime target; credential-free memory mode remains a development fallback.
- Vercel is the planned deployment host; deployment evidence is not yet recorded.
- Server-only secrets protect Twitch, Supabase, and provider credentials.
- Raw frames are ephemeral; raw chat is processed in memory unless approved for short-lived debugging; rewards are session-scoped and non-monetary.
- Third-party disclosures cover runtime dependencies, development tooling, Twitch, OBS, Supabase, Vercel, and provider status.

Evidence labels disclosures and source checks [diagnostic]; cloud/deployment proof [unknown].

Asset note Deployment health screenshot only after real preview is configured; no secret-bearing screenshots.

Final Deck Assets Must Prove The Golden Workflow Or Label The Gap

- Required final proof: real Twitch activity, real OBS-captured gameplay, exactly three validated quests, one streamer, two viewers, one authoritative revision, OBS overlay,
- Current manifest does not yet prove the real golden workflow.
- The proposal deck can be submitted honestly as a build-and-evidence plan; the final competition deck should swap unknown labels for manifest-backed real entries only
- Never upgrade fixture screenshots, static checks, or source inspection into live product claims.

Evidence labels final golden workflow [unknown]; available manifest evidence [fixture] and [diagnostic].

Asset note Privacy-reviewed demo recording stills, evidence manifest IDs, golden run limitations, and final source commit.